

Enhancing Traffic Sign Detection: YOLO and GAN Powered Performance Boost



A thesis submitted for the partial fulfillment of the requirements of the Bachelor of Science degree in Electrical and Electronic Engineering at Khulna University of Engineering & Technology, Khulna, Bangladesh

Submitted By
Pritom Karmoker
Roll: 1803108

Supervised By
Dr. Mohiuddin Ahmad
Professor

Electrical and Electronic Engineering
Khulna University of Engineering &
Technology

Department of Electrical and Electronic Engineering
Khulna University of Engineering & Technology
(KUET)
Khulna-9203, Bangladesh

Declaration

This is to certify that the thesis work, **Enhancing Traffic Sign Detection: YOLO and GAN Powered Performance Boost** was performed under the supervision of Prof. Dr. Mohiuddin Ahmad in the Department of Electrical and Electronic Engineering, Khulna University of Engineering & Technology, Khulna, Bangladesh. The above thesis work has not been submitted anywhere for any degree.

Signature of Supervisor

Signature of Candidate

Dedicated to
My
Beloved Parents

Acknowledgement

I would like to express my heartfelt gratitude for the opportunity to complete my thesis as an integral part of my Bachelor of Science in Electrical and Electronic Engineering degree at Khulna University of Engineering and Technology (KUET), Khulna. I am truly thankful for the blessings and kindness bestowed upon us by the Almighty, as without His desire, we would not have achieved our current position. I would like to express my sincere thanks and deep gratitude to my supervisor Dr. Mohiuddin Ahmad, Professor of the Department of Electrical & Electronic Engineering, for his unwavering support, patient guidance, and enthusiastic encouragement throughout the entire process of developing this thesis. His valuable feedback and scholarly advice have played a significant role in shaping this work into its final form. I am also grateful for the time and effort he dedicated to mentoring me, allowing me the space to explore and think critically. Furthermore, I extend my sincere appreciation to my parents for their continuous support and encouragement throughout my academic journey. Their belief in me has been a constant source of motivation, and I am truly grateful for their love and guidance.

Abstract

In the domain of traffic sign detection, the efficiency and accuracy of YOLOv4 and YOLOv5 algorithms make them indispensable. Despite their similar detection techniques, their considerable variance in parameters impacts detection speed. A significant challenge in this domain is the scarcity of data leading to overfitting and diminished accuracy. Addressing this, our research delves into the potential of Generative Adversarial Networks (GANs), specifically LSGAN and ProGAN, to expand datasets. In our study, we found that ProGAN generated images have higher similarity to real images compared to LSGAN-generated ones, based on SSIM and MSE values. Leveraging these images, we used an enhanced super-resolution approach with EDSR GAN to create higher-sized images and then compressed them. We mixed these differently sized images to train model, aiming to improve object detection robustness for various image sizes. Evaluating YOLOv4 and YOLOv5 models trained with real images alone, real images with LSGAN-generated images, and real images with ProGAN-generated images, we found that the YOLOv5 model trained with real and ProGAN-generated images achieved the highest accuracy and best detection performance. This highlights the effectiveness of integrating GAN-generated data and multi-scale training in enhancing traffic sign detection systems.

Table of Contents

Declaration.....	ii
Acknowledgement	iv
Abstract	v
List of Figures	viii
Chapter 1 Introduction.....	1
1.1 Overview	1
1.2 Motivation	3
1.3 Objectives.....	3
1.4 Thesis Layout	4
Chapter 2 Related Works	5
Chapter 3 Methodology	12
3.1 Operational framework	12
3.1.1 Yolo Object Detection.....	12
3.1.2 GAN Image Generation.....	14
3.2 Model Architecture.....	16
3.2.1 YOLOv4	16
3.2.2 YOLOv5	19
3.2.3 LSGAN.....	21
3.2.4 ProGAN.....	24
3.3 Experimental Setup	26
3.3.1 Image Dataset	26
3.3.2 Data Generation by LSGAN and ProGAN:	27
3.3.3 Multi-scale Image Generation	28
3.3.4 Formation of training dataset.....	29
3.3.5 YOLOv4 Model Setup.....	29
3.3.6 YOLOv5 Model Setup.....	30

3.3.7 System Overview	30
Chapter 4 Result Analysis	32
4.1 Data Generation Result	32
4.2 YOLOv4 and YOLOv5 Training Results	41
4.2.1 Performance Metrics	41
4.2.2 Performance Comparison	44
4.2.3 User Interface	47
Chapter 5 Conclusion and Future Work	48
5.1 Conclusion.....	48
5.2 Future Work.....	49

List of Figures

Figure No.	Description of Figure	Page No.
Figure 3.1:	Yolo basic architecture	13
Figure 3.2:	Generative Adversarial Network	14
Figure 3.3:	YOLOv4 object detector	16
Figure 3.4:	YOLOv5 Architecture	19
Figure 3.5:	Generator and Discriminator Architecture of LSGAN	21
Figure 3.6:	Progressive growing architecture of ProGAN	24
Figure 3.7:	Selected dataset categories (a) II-2, (b) II-4, (c) II-30-50, (d) II-34, (e) III-107.1-2	27
Figure 3.8:	An overview of the system	31
Figure 4.1:	Original and generated images (a) Original Images, (b) LSGAN generated images, and (c) ProGAN generated images	36
Figure 4.2:	LSGAN generated images with size (a) 32×32 , (b) 64×64 , (c) 128×128 , and (d) 256×256	38
Figure 4.3:	ProGAN generated images with size (a) 32×32 , (b) 64×64 , (c) 128×128 , and (d) 256×256	39
Figure 4.4:	(a) LSGAN and (b) ProGAN generated images integrated on real training image scenes	40
Figure 4.5:	The detection result for (a) YOLOv4 and Group 1, (b) YOLOv4 and Group 2, and (c) YOLOv4 and Group 3 dataset and for (d) YOLOv5 and Group 1, (e) YOLOv5 and Group 2, and (f) YOLOv5 and Group 3 dataset	46
Figure 4.6:	User interface for detecting traffic sign in an image	47

Chapter 1

Introduction

1.1 Overview

In the realm of computer vision and intelligent transportation systems, the accurate detection of traffic signs stands as a critical component for ensuring road safety and efficient navigation. It prevents accidents by informing drivers of speed limits and rules, guides autonomous vehicles, protects pedestrians, aids urban planning, and improves emergency response.

Deep neural networks, a subset of machine learning, revolutionize object detection by autonomously learning intricate features from raw data, drastically enhancing detection accuracy and reliability [1,2,3]. In object detection, Convolutional Neural Networks (CNNs) are indispensable, employing hierarchical feature extraction to precisely localize and categorize objects within images, while in traffic sign detection, CNNs analyze visual data to automate recognition, advancing systems for road safety and autonomous driving [4,5]. Initially, Convolutional Neural Networks (CNNs) were good at spotting traffic signs but not at pinpointing their exact locations. To fix this, Faster R-CNN [6] and SSD [7] were created to detect both objects and their positions. SSD and Faster R-CNN are improvements over CNNs because they integrate object location into the network. SSD scans different scales for objects, while Faster R-CNN speeds up detection by suggesting regions efficiently. These advancements enable better object recognition and precise location, going beyond traditional CNNs. However, SSD struggles with detecting small objects accurately because of its fixed anchor boxes, and Faster R-CNN can be slow due to its multi-step region proposal process.

YOLO (You Only Look Once) addresses these limitations by being faster and more efficient. It divides the image into a grid and predicts bounding boxes and class probabilities directly, allowing real-time object detection. YOLO's advantages lie in its speed, simplicity, and ability to detect objects in a single pass, making it suitable for applications requiring speed and accuracy. The evolution of YOLO models showcases a remarkable progression in real-time object detection [8]. You Only Look Once laid the foundation by introducing a unified framework for efficient real-time detection. Subsequently, YOLO small object detection addressed the challenge of detecting small objects, enhancing the model's versatility and applicability [9]. YOLO9000 further extended the capabilities by introducing a large-scale dataset and a hierarchical classification system, facilitating the detection and classification of

a vast array of object categories [10]. Then, YOLO traffic sign detection signifies a specialized application, demonstrating the adaptability of YOLO for specific tasks such as traffic sign recognition, showcasing its effectiveness across diverse domains [11]. YOLOv3 improves accuracy and detection performance through a deeper backbone network, multi-scale detection, and better bounding box refinement compared to previous YOLO models.

Object detection got a big boost with YOLOv4 and YOLOv5. YOLOv4 speed things up and got more accurate with CSPDarknet53 [14]. YOLOv5 focused on making models efficient for training and spotting stuff. Newer versions might not always be better due to issues like not improving much or not fitting well with what's already set up. In real-world use, developers pick between YOLOv4 or YOLOv5 based on how well they balance performance, speed, and fitting into systems.

The problem of not having enough labeled data for training YOLOv4 and YOLOv5 models in traffic sign detection leads to overfitting. This means that the models become too specialized in recognizing the limited examples they were trained on, making them less accurate when dealing with new, unseen situations. Another issue is that the dataset used to train these models often lacks objects of different sizes. This means that the models are mainly taught to detect objects that are all roughly the same size. As a result, when they encounter objects of various sizes in real-life scenarios, they struggle to accurately identify them.

The problem of Limited Labeled Data can be mitigated by generating images using Generative Adversarial Networks (GAN) [15,16]. DCGAN (Deep Convolutional Generative Adversarial Network) and WGAN (Wasserstein Generative Adversarial Network) are both advancements in the realm of Generative Adversarial Networks (GANs), with DCGAN [17] utilizing convolutional layers for image generation and WGAN [18] introducing the Wasserstein distance metric for more stable training. Progressive GAN represents a further evolution, focusing on progressive growing techniques to generate high-resolution images with enhanced realism and detail, pushing the boundaries of generative modeling [19]. However, GAN-generated images may exhibit significant disparities compared to real ones, posing a challenge to their effectiveness. To check how close they are, we can use SSIM [20] and MSE. Higher SSIM and lower MSE in the new dataset mean the pictures are more like the real ones. This helps the model be more accurate. Notably, LSGAN [21] and ProGAN [22] are distinguished among GANs for prioritizing improved image quality and training stability.

The challenge of Limited Variability in Object Sizes can be addressed by generating and training the model with images of varying sizes. Lower-sized images can be obtained through simple resizing techniques, while higher resolution images can be generated using the EDSR GAN model [23].

The incorporation of GAN-generated data combined with multi-scale training methods significantly improves the effectiveness of both YOLOv4 and YOLOv5 models in detecting traffic signs.

1.2 Motivation

Navigating modern roads demands autonomous driving and enhanced safety, making precise traffic sign detection crucial. Conventional CNN methods in this field employ multi-stage architectures for traffic sign detection, often leading to increased computational complexity. However, conventional methods struggle with real-world complexities such as varying lighting and occlusions. Herein lies our challenge: to elevate traffic sign detection beyond limitations. The fusion of YOLO's real-time object detection and GAN's synthetic data generation emerges as a transformative solution. By harnessing YOLO and GAN technologies, we can equip systems to swiftly interpret and respond to traffic signs, even in adverse conditions. This integration promises improved accuracy, adaptability, and efficiency in traffic sign recognition. In this manner, the embrace of this innovative path can take place, reshaping road safety through YOLO and GAN-powered enhancements, leading to a future of driving that is both safer and smarter.

1.3 Objectives

Our research aims to enhance traffic sign detection using YOLOv4 and YOLOv5 algorithms. Challenges such as parameter variance affecting detection speed and data scarcity leading to overfitting are addressed by exploring Generative Adversarial Networks (GANs) like LSGAN and ProGAN to generate additional data. Therefore, the outlines of the research objectives are to

1. enhance YOLO's traffic sign detection by integrating GAN-generated images, compensating for the limitations of a small dataset.

2. investigate how GAN-generated images improve YOLO's sign detection across diverse scenarios and counteract limited dataset impact through GAN-based data augmentation for YOLO's performance enhancement.
3. alleviate dataset bias by including various object sizes in training data to enhance YOLOv4 and YOLOv5 algorithms' detection across a broader size range
4. measure the performance boost achieved by YOLO when fueled by GAN-generated images, focusing on accuracy and recall.
5. showcase YOLO's enhanced real-world performance in challenging traffic sign detection scenarios, highlighting the practical value of the GAN-powered approach.

1.4 Thesis Layout

Chapter 1: Introduction

Introduces research background and motivation in traffic sign detection, reviews previous works, explains the significance of YOLO algorithms, discusses challenges of data scarcity, proposes GAN-based approach for expanding datasets, and outlines thesis aims and objectives.

Chapter 2: Related work

The Related work chapter analyzes previous studies on traffic sign detection, highlighting their strengths, challenges, and impact on my current research.

Chapter 3: Methodology

Details methodological approach for YOLOv4 and YOLOv5 experiments, demonstrates integration of GAN-generated data and multi-scale training.

Chapter 4: Result Analysis

Analyzes results of integrating GAN-generated data and multi-scale training impact on YOLO's performance, and compares models trained with different datasets.

Chapter 5: Conclusion and Future Work

Summarizes key findings and outcomes, discusses effectiveness of GAN-generated data and multi-scale training, suggests future research areas and improvements in methodology.

Chapter 2

Related Works

Table 2.1: Related works

Ref.	Proposal	Input	Output	Dataset	Problem	Method	Result	Limitations
[25]	Detecting small traffic signs in realistic scenes for driver assistance systems and autonomous driving	Images from a camera capturing the traffic scene	The real-time detection and localization of small traffic signs in realistic images	TT100K dataset.	Detecting small traffic signs in realistic scenes for driver-assistance systems and autonomous driving	MSA YOLOv3 uses a multi-scale spatial pyramid pooling block and image mixup technology	Detection speed 23.81 FPS and mAP of 86% on the TT100K dataset	MSA_YOLO v3 greatly increased computational complexity so detection speed is less
[26]	Compressed YOLOv4 model: Lightweight, smaller size, minimal accuracy loss, ideal for edge device deployment	The model takes images as input to perform object detection.	Images with detected object having bounding box coordinates, object class and confidence score	SAR Ship Detection Dataset (SSDD) and Gaofen Airplane Dataset	Reduced model accuracy, potential information loss, complexity in optimization, compatibility constraints, and specific hardware dependency	They used sparsity training, pruning, knowledge distillation, quantization, and edge deployment to compress YOLOv4	Achieves a compressed Light-YOLOv4 model with speed $4.2\times$ and compromising mAP 0.013	Accuracy reduction due to aggressive compression and potential challenges in generalization across diverse scenarios and hardware

Ref.	Proposal	Input	Output	Dataset	Problem	Method	Result	Limitations
[27]	This paper proposes improving traffic sign recognition by generating synthetic training data with GANs and assessing its quality	Original images and synthetic images	Improved traffic sign recognition performance	Taiwan Prohibitory Sign	Limited availability of annotated training data for traffic signs	DCGAN, WGAN and LSGAN for synthetic image generation. YOLOv4 and YOLOv5 for recognition	Accuracy achieved was 84.9% on Yolo V3 and 89.33% on Yolo V4	Lack of comparison of the performance of different GAN models in generating synthetic images for traffic sign recognition
[28]	An enhanced light-weight traffic sign recognition algorithm based on YOLOv4-Tiny, addressing low detection and positioning accuracy	An image contains a scene from a road or traffic environment	Detected traffic sign images with bounding box and class label information	Tsinghua-Tencent 100K	Low detection accuracy, inaccurate positioning accuracy, small target detection and high overlapping targets	YOLOv4-Tiny with improved anchor generation, large scale feature map optimization and enhanced non maximum suppression	mAP and recall are improved by 5.73% and 7.29% respectively, and FPS value is maintained at about 87 f/s	The potential lack of generalization to diverse datasets and real-world conditions, and the specific improvements' sensitivity to anchor generation and small target variability

Ref.	Proposal	Input	Output	Dataset	Problem	Method	Result	Limitations
[29]	A deep model for traffic sign detection and recognition that addresses challenges such as failure detection, target size, and occlusion	An image representing a scene from a road or traffic environment	Image with bounding box and class label on each traffic sign	Complex Conditions Traffic Sign Detection Benchmark 2021	Failure detection, target size and localization, overlapping and occlusion and varying target sizes	Coordinate Attention, angle loss, SimOTA label assignment, and hierarchical -path feature fusion on base YOLOv5	High detection speed of 91 FPS. Precision reaches 98.1%, recall reaches 97.6%, and mAP@0.5 reaches 98.8%	Limited examination of model's real-world generalization and potential biases in training data impacting performance on new scenarios
[30]	Lightweight model improves YOLOv5: higher accuracy, fewer parameters, enhanced attention, and features	The input of the proposed model is typically an image	The model's output consists of detected object bounding boxes, class labels, and confidence scores for each object	VOC2007 dataset	Achieving higher accuracy through reduced parameters, improved attention mechanisms, and enhanced feature extraction	Lightweight YOLOv5 with SKConv, CECA, and MSM, and deeper network.	A reduction in parameters and computation, achieving a significant increase in mAP@0.5 and mAP@0.5:0.95 on the VOC2007 test dataset	The model's speed is compromised due to memory access time costs caused by the use of group convolution and multi-branch computation

Ref.	Proposal	Input	Output	Dataset	Problem	Method	Result	Limitations
[31]	A comprehensive review of GAN models, mechanisms, applications, and challenges in computer vision	Grayscale images	Generated grayscale images	MNIST and Fashion-MNIST	In-depth GAN analysis in computer vision: theory, models, apps, challenges, empirical findings, and future research directions	DCGAN, LPCAN, SAGAN, pix2pix, CycleGAN, DiscoGAN, DTN, Sem-GAN, StackGAN, GAN-CLS, RTT-GAN, TP-GAN, PGGAN	The best FID value obtained on MNIST is 1.92 and on fashion MNIST dataset is 5.76 both using WGAN-GP	Limited evaluation system for GAN models and potential for further theoretical advancements and broader applications are notable limitations
[32]	Using DCGAN, to augment tomato leaf disease images, leading to improved recognition accuracy and generalization for disease identification.	The model takes original and DCGAN generated tomato leaf disease images	Model combines original and DCGAN-generated images for improved accuracy and generalization in recognition tasks	Leaf disease images from GoogLeNet	Data augmentation technique employing DCGAN-generated images to improve tomato leaf disease recognition	Uses DCGAN for data augmentation and GoogLeNet for tomato leaf disease recognition	The proposed method achieves a top-1 average identification accuracy of 94.33% for tomato leaf disease recognition	The limitations of this work include potential sensitivity to hyperparameters and the need for selecting appropriate generative adversarial networks for optimal performance

Ref.	Proposal	Input	Output	Dataset	Problem	Method	Result	Limitations
[33]	Introduces LSGANs with least squares loss for better image quality, stable training, and reduced vanishing gradients	Noise for generator and real images for discriminator	A generated image, that aims to closely resemble real data instances from a given dataset	The SVHN, LSUN-bedroom, and cat datasets	This work introduces LSGANs with a novel least squares loss, enhancing image quality, stability, and mitigating vanishing gradients	This work uses LSGANs with least squares loss for both discriminat or and generator, validated through experiments on diverse datasets	LSGANs using LSGANs (-110) parameters outperform LSGANs (011) in speed and performance on SVHN, LSUN-bedroom, and cat datasets	The limitations of this work include a focus on stability and image quality improvements without exploring alternative strategies for enhancing generated sample proximity to real data
[34]	GAN-Based Day-to-Night Image Style Transfer for Nighttime Vehicle Detection	The input of the model is a daytime image of a scene containing vehicles on the road	The output of the model is a nighttime version of the input image	SYNTHIA, KITTI, GTA and ITRI-night datasets	GAN enhances nighttime vehicle detection by transforming daytime images into nighttime style while preserving vehicle attributes	AugGAN, CycleGAN and UNIT for image style transfer. Faster-R-CNN and YOLO network for detection	GTA-val-D2N data has highest degree of object preservation with MOS value 3.91 when learning from GTA	Uni-modal nighttime image generation, limited handling of complex domain shifts, potential artifact presence

Ref.	Proposal	Input	Output	Dataset	Problem	Method	Result	Limitations
[35]	Synthesizing traffic sign images using Generative Adversarial Networks (GANs).	Standard traffic sign template, background image patch.	Realistic synthetic traffic sign images.	German Traffic Sign Recognition Benchmark (GTSRB).	Limited availability of labeled training data for traffic sign classification .	Conditional Generative Adversarial Networks (cGANs) incorporating background images and standard sign templates.	Improved classification accuracy and visually realistic synthesis results compared to traditional methods.	Synthesized images may still need further improvement to match real traffic sign images.
[36]	Introduces the GTSRB dataset and presents IJCNN 2011 competition results.	Over 50,000 traffic sign images captured in various conditions.	Classifies images into 43 traffic sign classes.	GTSRB: 50,000+ labeled traffic sign images across 43 classes.	Recognizing traffic signs despite variations in visual appearance.	Participants used CNNs, SVMs, LDA, and precomputed features.	Over 20 teams achieved high accuracies comparable to humans.	Unbalanced class frequencies, visual appearance variations, and potential biases in competition protocol.

Ref.	Proposal	Input	Output	Dataset	Problem	Method	Result	Limitations
[37]	Develop a real-time detection system for Taiwan stop signs using YOLO V3 with optimized anchor calculation.	Image data of Taiwan stop signs.	Efficient and accurate real-time detection system for Taiwan stop signs.	Custom dataset of Taiwan stop sign images.	Need for efficient detection of Taiwan stop signs considering varying visual appearances and environmental conditions.	Utilize YOLO V3 with k-means based anchor calculation for optimized performance.	Improved accuracy and reduced processing time compared to baseline configurations.	Lack of pre-existing dataset for Taiwan stop signs, potential for further enhancements with GAN integration and higher-dimensional images.
[38]	Develop a CNN for simultaneous traffic sign detection and classification in real-world images.	Real-world images with varying illuminance and weather conditions containing traffic signs.	Accurate detection and classification of traffic signs.	Tsinghua-Tencent 100K benchmark with 100,000 annotated images.	Traditional methods lack simultaneous solutions for small traffic sign detection and classification.	Two-phase approach using a CNN architecture.	Achieved 84% accuracy and 94% recall in traffic sign detection, outperforming traditional methods.	Challenges in balancing class distributions during training and addressing real-world variations.

Chapter 3

Methodology

3.1 Operational framework

Object detection is a fundamental computer vision task that involves identifying and localizing specific objects within images or video frames. It goes beyond image classification by not only recognizing what objects are present, but also precisely locating them with bounding boxes. This technology finds applications in diverse fields such as autonomous driving, surveillance, retail analytics, and more, enabling machines to perceive and interact with their surroundings in a detailed and contextually aware manner.

3.1.1 Yolo Object Detection

YOLO, or "You Only Look Once" is a fast and efficient object detection algorithm in computer vision. It directly predicts bounding boxes and class probabilities in a single pass of a neural network, enabling real-time object detection in images and videos. Figure 3.1 shows the Yolo basic architecture.

Simplified overview of Yolo object detection technique:

Input Image: The algorithm takes an input image as its initial input.

Divide Image into Grid: YOLO divides the input image into a grid of cells. The grid can be of any size, but typically, it's $S \times S$ where S is a hyperparameter.

Object Localization: For each grid cell, YOLO predicts bounding boxes. Each bounding box consists of 5 components: (x, y) coordinates of the bounding box's center relative to the grid cell, width w and height h of the bounding box relative to the entire image, and a confidence score indicating the probability that the bounding box contains an object.

Object Classification: YOLO also predicts the probability of each bounding box containing different classes of objects. This is done using softmax activation function to get class probabilities.

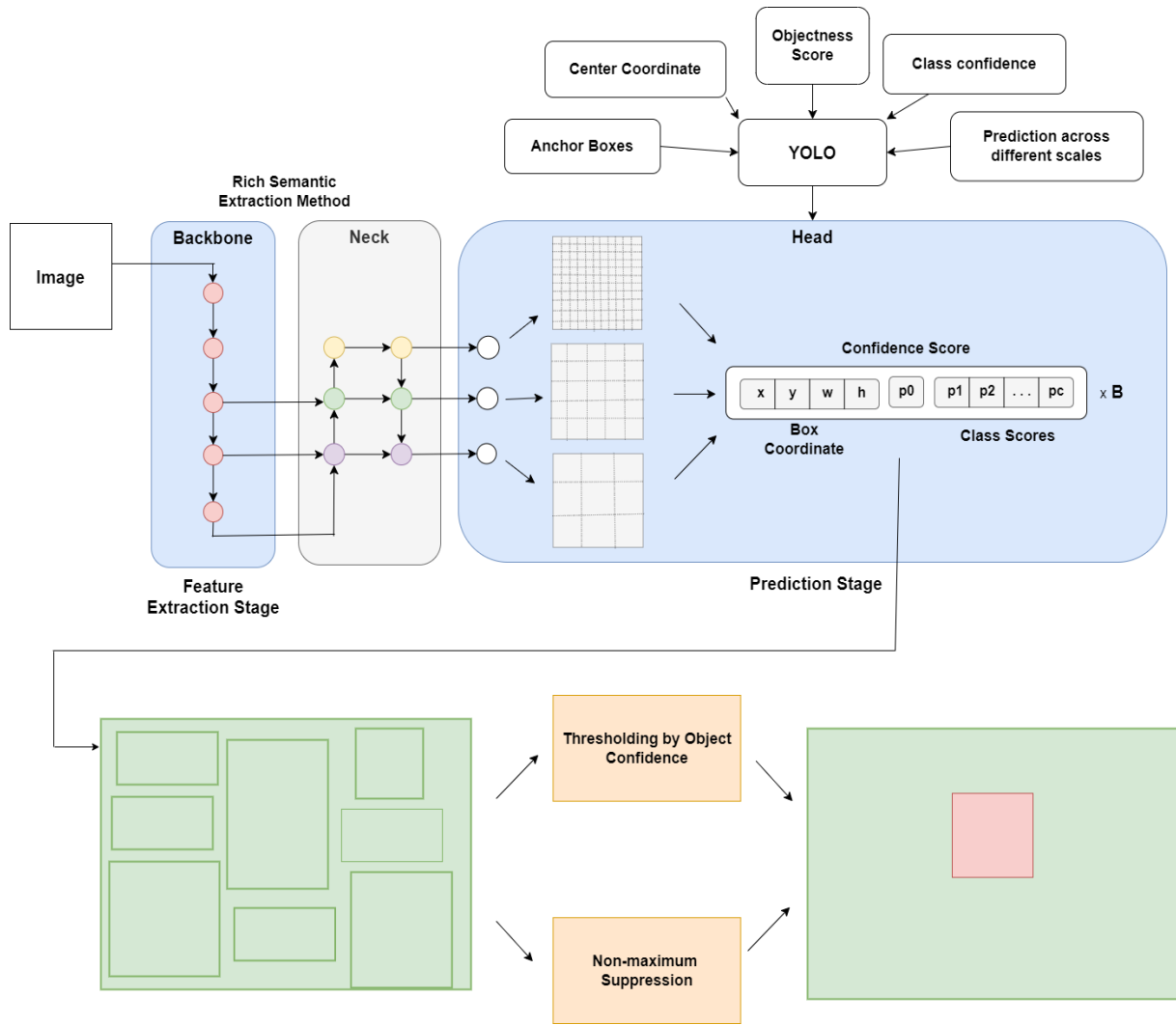


Figure 3.1: Yolo basic architecture.

Bounding Box Selection: After predictions are made for all grid cells, YOLO applies a confidence threshold to filter out bounding boxes with low confidence scores.

Non-Maximum Suppression (NMS): To eliminate redundant bounding boxes for the same object, YOLO uses non-maximum suppression. This involves sorting the bounding boxes based on their confidence scores and iteratively selecting the ones with the highest scores while suppressing (removing) highly overlapping bounding boxes.

Output: The final output of the YOLO algorithm is a set of bounding boxes along with their associated class labels and confidence scores, representing the detected objects in the input image.

3.1.2 GAN Image Generation

A Generative Adversarial Network (GAN) is a class of artificial intelligence algorithms used in unsupervised machine learning. The core principle of a GAN involves two neural networks – the generator and the discriminator – which are trained simultaneously through a competitive process.

Generator: The generator takes random noise (typically drawn from a Gaussian distribution) as input and generates data samples (e.g., images) from this noise. The goal of the generator is to produce synthetic data that is indistinguishable from real data.

Discriminator: The discriminator is like a binary classifier. It receives input data samples (either real or generated by the generator) and classifies them as either real or fake (generated). The discriminator's objective is to correctly classify real data as real and fake data as fake.

Figure 3.2 illustrates the collaborative effort of the generator and discriminator in producing synthetic images.

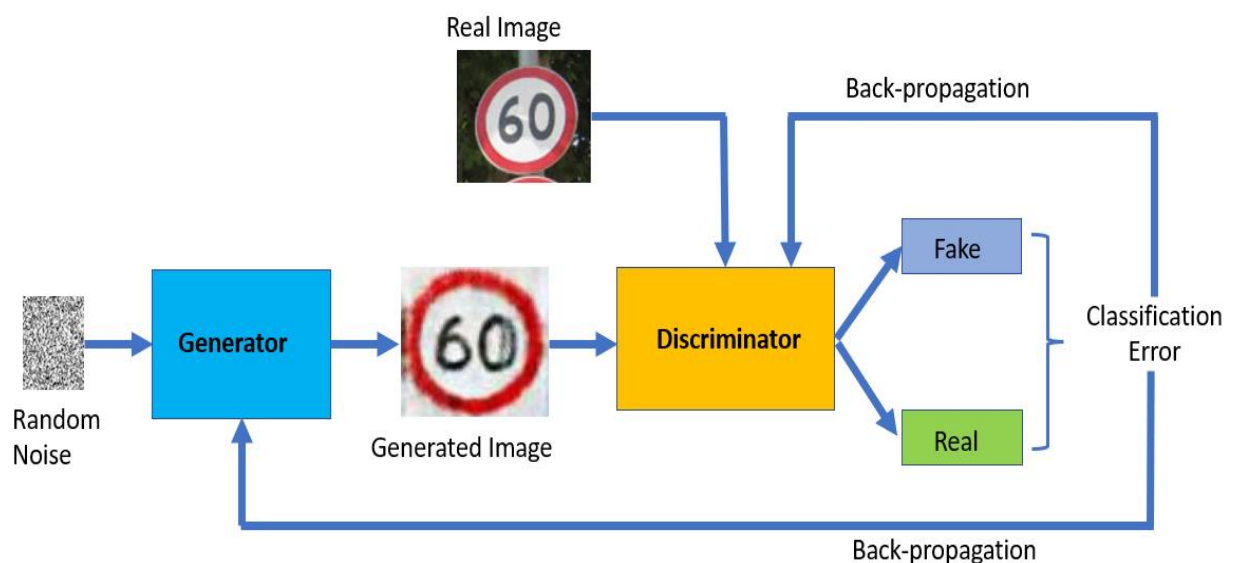


Figure 3.2: Generative Adversarial Network.

Training Process:

1. Initially, both the generator and discriminator are initialized with random weights.
2. During training, the generator generates fake data samples from random noise, while the discriminator evaluates these samples and provides feedback.
3. The discriminator is trained using a combination of real data samples and fake data samples generated by the generator. It learns to differentiate between real and fake data.
4. Meanwhile, the generator is trained to improve its ability to generate realistic data by receiving feedback from the discriminator. The generator's objective is to produce data samples that can fool the discriminator into classifying them as real.
5. This process of competition between the generator and discriminator continues iteratively, with both networks getting better over time.

Loss Function: The training process involves optimizing a loss function that captures the performance of both the generator and discriminator. The generator aims to minimize this loss function by generating more realistic samples, while the discriminator aims to maximize it by correctly classifying real and fake samples.

Convergence: Ideally, the training process continues until the generator produces data samples that are indistinguishable from real data, and the discriminator cannot reliably differentiate between real and fake samples. At this point, the GAN is said to have converged.

GANs leverage a competitive process between a generator and a discriminator to learn to generate synthetic data that closely resembles real data, without the need for explicit supervision.

Figure 3.2 shows the functioning of generator and discriminator in a generative adversarial network.

3.2 Model Architecture

This section provides an overview of the model architecture utilized in our work, focusing on enhancing traffic sign detection through the integration of pre-existing models, specifically You Only Look Once (YOLO) and Generative Adversarial Network (GAN) techniques. The models discussed herein have been developed by previous researchers and serve as foundational components in our approach to improving traffic sign detection accuracy and performance.

3.2.1 YOLOv4

YOLOv4 is an enhanced version of the YOLO (You Only Look Once) object detection algorithm, aiming for an optimal balance between speed and accuracy. Figure 3.3 displays the YOLOv4 architecture including backbone, neck and head.

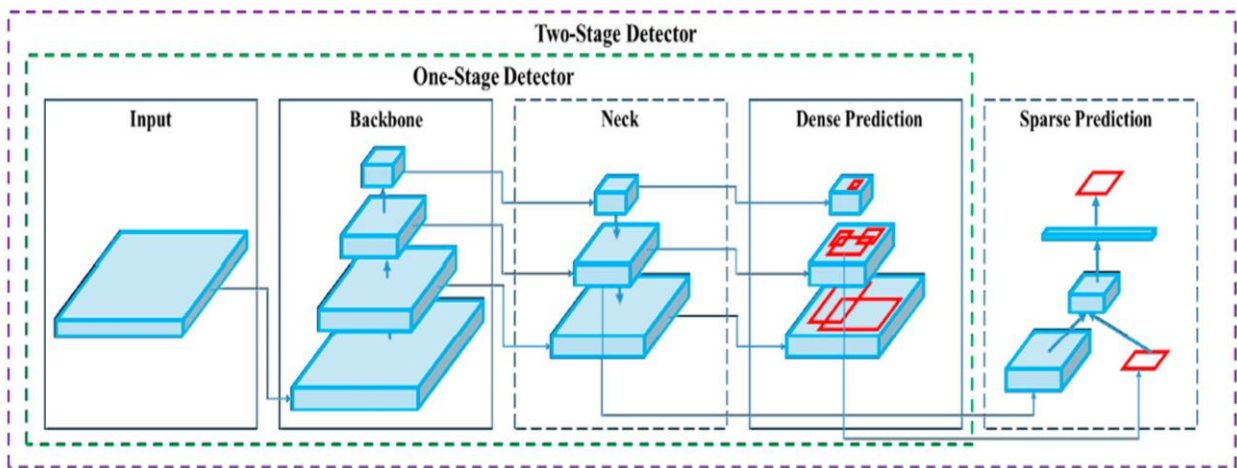


Figure 3.3: YOLOv4 object detector.

Input: The input layer of YOLOv4 takes an image of fixed dimensions (e.g., 416x416 pixels) as input.

Backbone: The backbone of YOLOv4 consists of a convolutional neural network (CNN) that extracts features from the input image.

Main architectural blocks:

- CSPDarknet53: A variant of the Darknet backbone with Cross Stage Partial connections for feature reuse and better gradient flow.

- CSPResNeXt: A residual network with Cross Stage Partial connections and cardinality, enhancing feature representation and diversity.

Neck: The neck of YOLOv4 is responsible for combining features from different scales and levels of abstraction.

Main architectural block:

- Spatial Pyramid Pooling (SPP): A pooling layer that captures features at multiple scales by applying pooling operations with different kernel sizes.

Dense Prediction: The dense prediction module predicts dense feature maps at multiple scales, which are then used for object detection.

Main architectural blocks:

- PANet (Path Aggregation Network): A network that aggregates features from different levels of the feature pyramid using a top-down path and lateral connections.
- YOLOv3 Head: The detection head used in previous versions of YOLO, consisting of convolutional layers for predicting bounding boxes, objectness scores, and class probabilities.

Sparse Prediction: The sparse prediction module predicts sparse feature maps at a single scale, which are used for object detection in conjunction with the dense predictions.

Main architectural block:

- YOLOv4 Head: A modified version of the YOLOv3 head that introduces additional convolutional layers and techniques like mish activation and PANet-like connections for improved performance.

Head: The head of YOLOv4 combines the dense and sparse predictions to generate the final set of bounding boxes, objectness scores, and class probabilities.

Main architectural block:

- YOLO Detection Layer: A specialized layer that processes the dense and sparse predictions to produce the final detection results.

One-Stage Detector: YOLOv4 is primarily a one-stage detector, meaning it directly predicts bounding boxes and class probabilities without separate proposal generation and refinement stages.

Two-Stage Detector: Although YOLOv4 is a one-stage detector, it incorporates elements like PANet and dense predictions that borrow concepts from two-stage detectors, enhancing its performance and robustness.

These architectural blocks constitute the backbone of the YOLOv4 model architecture, enabling efficient and accurate object detection in real-time scenarios.

The key improvements in YOLOv4 over previous models:

- **Enhanced Backbone Architecture:** YOLOv4 adopts CSPDarknet53-PANet-SPP, integrating cross-stage partial connections (CSP) in Darknet-53 to reduce computation while maintaining accuracy.
- **Bag of Freebies (BoF) Integration:** Various data augmentation techniques such as mosaic augmentation and DropBlock regularization are implemented to improve model generalization. Additionally, class label smoothing, CIoU loss, and cross-mini-batch normalization (CmBN) are utilized during training for enhanced performance.
- **Self-adversarial Training (SAT):** YOLOv4 employs an adversarial attack on input images during training to enhance model robustness against perturbations, ensuring accurate object detection in challenging scenarios.
- **Hyperparameter Optimization with Genetic Algorithms:** Genetic algorithms are utilized to optimize hyperparameters during training, coupled with a cosine annealing scheduler to dynamically adjust the learning rate, leading to improved convergence and performance.

Overall, YOLOv4 combines advancements in architecture, training approaches, regularization techniques, and hyperparameter optimization to achieve significant improvements in object detection accuracy while maintaining real-time performance.

3.2.2 YOLOv5

YOLOv5, an evolution of the You Only Look Once (YOLO) algorithm, emphasizes an ideal blend of speed and accuracy, refining object detection capabilities through advanced architecture and methodology.

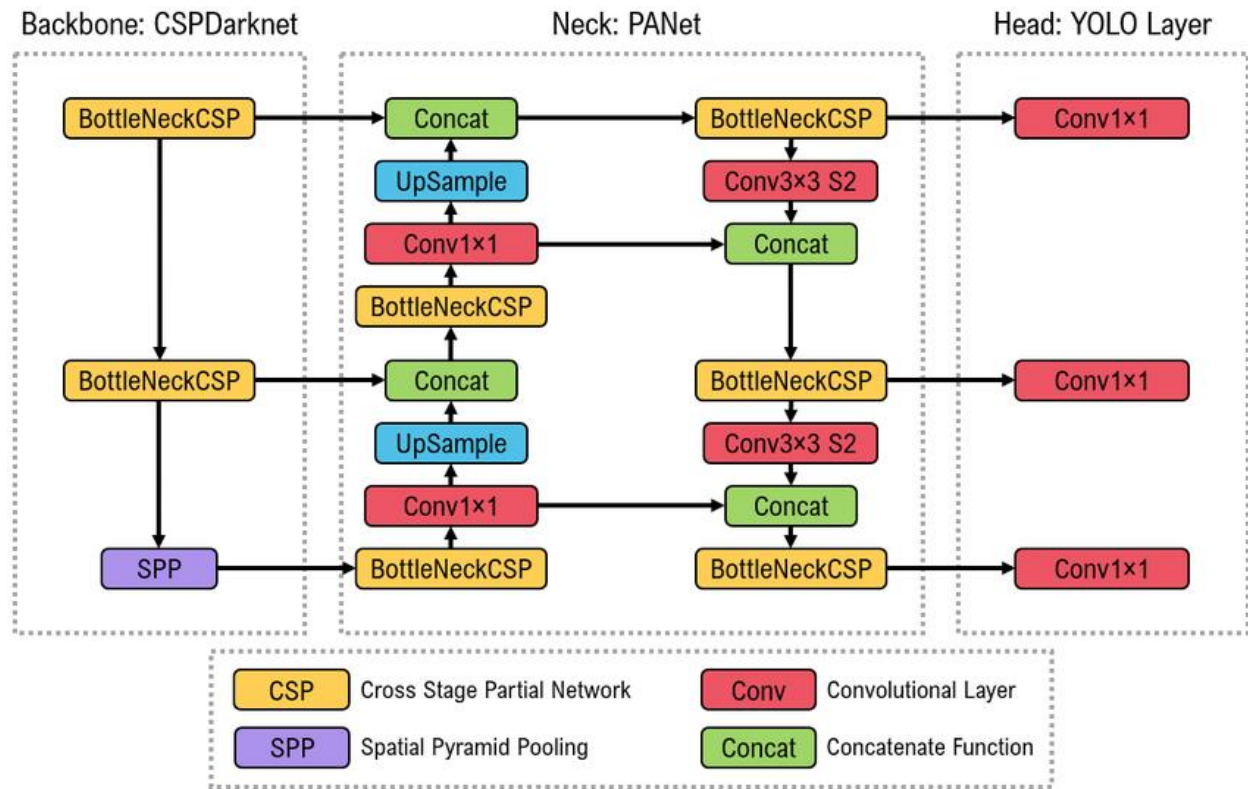


Figure 3.4: YOLOv5 Architecture.

Backbone: CSP-Darknet53

- YOLOv5 employs CSP-Darknet53 as its backbone, which is essentially the convolutional network Darknet53 used in YOLOv3 but with the addition of the Cross Stage Partial (CSP) network strategy.
- CSPNet partitions the feature map of the base layer into two parts and then merges them through a cross-stage hierarchy, reducing the number of parameters and computational requirements.

Neck: SPP and PANet

YOLOv5 introduces two significant changes to the model neck:

- Spatial Pyramid Pooling (SPP): An aggregation block that significantly increases the receptive field and segregates the most relevant context features without sacrificing network speed.
- Path Aggregation Network (PANet): A modified feature pyramid network incorporating the BottleNeckCSP module, which improves information flow and localization accuracy.

Head: Detection Head

- YOLOv5 utilizes the same detection head as YOLOv3 and YOLOv4, composed of three convolution layers that predict bounding box coordinates (x, y, height, width), objectness scores, and object classes.

Activation Function: SiLU and Sigmoid

- YOLOv5 employs the SiLU (Sigmoid Linear Unit) activation function in hidden layers and the Sigmoid activation function in the output layer, providing non-linearity and enabling effective feature representation.

Loss Function: BCE and CIoU

- YOLOv5 utilizes Binary Cross Entropy (BCE) to compute classes and objectness losses, and Complete Intersection over Union (CIoU) loss to compute location loss. The final loss is a combination of these components.

YOLOv5 introduces several architectural improvements over YOLOv4, including an efficient backbone network, enhanced feature fusion techniques, and novel data augmentation strategies. Additionally, YOLOv5 introduces a simpler and more flexible architecture, facilitating easier customization and deployment across various hardware platforms. These enhancements collectively contribute to improved accuracy and speed in object detection tasks compared to YOLOv4. The architecture of YOLOv5 demonstrated in Figure 3.4.

3.2.3 LSGAN

LSGAN architecture closely resembles that of traditional GANs, with a primary distinction lying in the usage of a least squares loss function instead of the sigmoid cross-entropy loss function. The discriminator's architecture remains largely identical to that of traditional GANs.

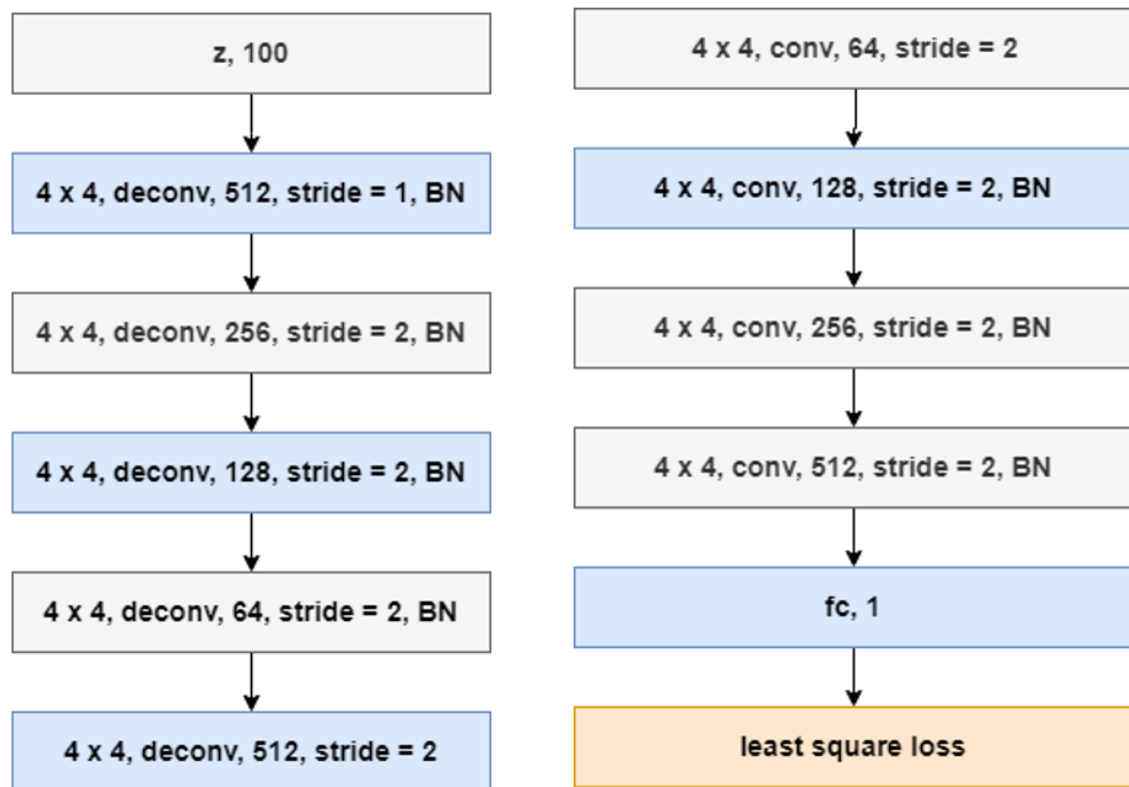


Figure 3.5: Generator and Discriminator Architecture of LSGAN.

LSGAN consist of two main block of specially connected convolution layer called generator and discriminator.

Generator (G):

- The generator takes random noise or a random vector z as input and generates synthetic samples.
- It typically consists of one or more hidden layers followed by an output layer.
- Spectral normalization is applied to the convolutional transpose layers in every layer of the generator.
- Each hidden layer in the generator utilizes batch normalization and ReLU activation function, except for the output layer.

- The output layer employs the Tanh activation function to map the input noise to the output space.

Discriminator (D):

- The discriminator is a binary classifier that aims to distinguish between real and fake samples.
- It takes samples from both the real data distribution and the generated samples from the generator as input.
- Similar to the generator, it typically consists of several hidden layers followed by an output layer.
- Spectral normalization is applied to the convolutional layers in every layer of the discriminator.
- Each hidden layer in the discriminator uses batch normalization and LeakyReLU activation function, except for the output layer.
- The final layer computes a probability score, using a fully connected layer and sigmoid activation, to determine if the input sample is real or fake.

Objective Function:

- LSGAN introduces a new objective function based on the least squares loss instead of the traditional binary cross-entropy loss used in standard GANs.
- The objective of the discriminator is to minimize the least squares loss between the predicted probabilities and the target labels (0 for fake, 1 for real).
- The objective of the generator is to minimize the least squares loss between the predicted probabilities of the discriminator and the target labels (1 for real).

The expression for the Least Squares Generative Adversarial Network (LSGAN) objective function can be represented through equation 1.

$$\min_G \max_D V_{LSGAN}(D, G) = E_{x \sim P_{data}(x)} [(D(x) - b)^2] + E_{z \sim P_z(z)} [(D(G(z)) - a)^2] \quad (1)$$

In the context of the Least Squares Generative Adversarial Network (LSGAN), several key components contribute to its objective function. The generator network, denoted as G , generates samples $G(z)$ from random noise z , with z drawn from a specified distribution $P_z(z)$. Real data samples x are drawn from the distribution $P_{data}(x)$. The discriminator network D , is tasked with distinguishing between real and generated samples. To stabilize training,

hyperparameters a and b are typically set to 0 and 1 respectively. The objective function $V_{LSGAN}(D, G)$ seeks to minimize the discrepancy between the discriminator's predictions on real and generated data, promoting the generation of realistic samples by the generator while maximizing the discriminator's ability to differentiate between real and fake samples.

Training Procedure:

- During training, the generator and discriminator are trained iteratively in an adversarial manner.
- The generator tries to generate samples that are indistinguishable from real samples, while the discriminator tries to correctly classify real and fake samples.
- The least squares loss is computed and used to update the weights of both the generator and discriminator using Adam optimizer.

Overall, LSGAN aims to improve the training stability and alleviate mode collapse issues in traditional GANs by using a different loss function based on least squares. It has been shown to produce higher-quality samples and achieve better convergence properties in certain applications compared to standard GANs. Figure 3.5 shows generator and discriminator architecture of LSGAN.

3.2.4 ProGAN

ProGAN or Progressive Growing of GANs, is a training methodology for Generative Adversarial Networks that gradually increases the resolution of generated images during training to improve stability and quality. It was introduced to address challenges in training large-scale GANs for generating high-resolution images.

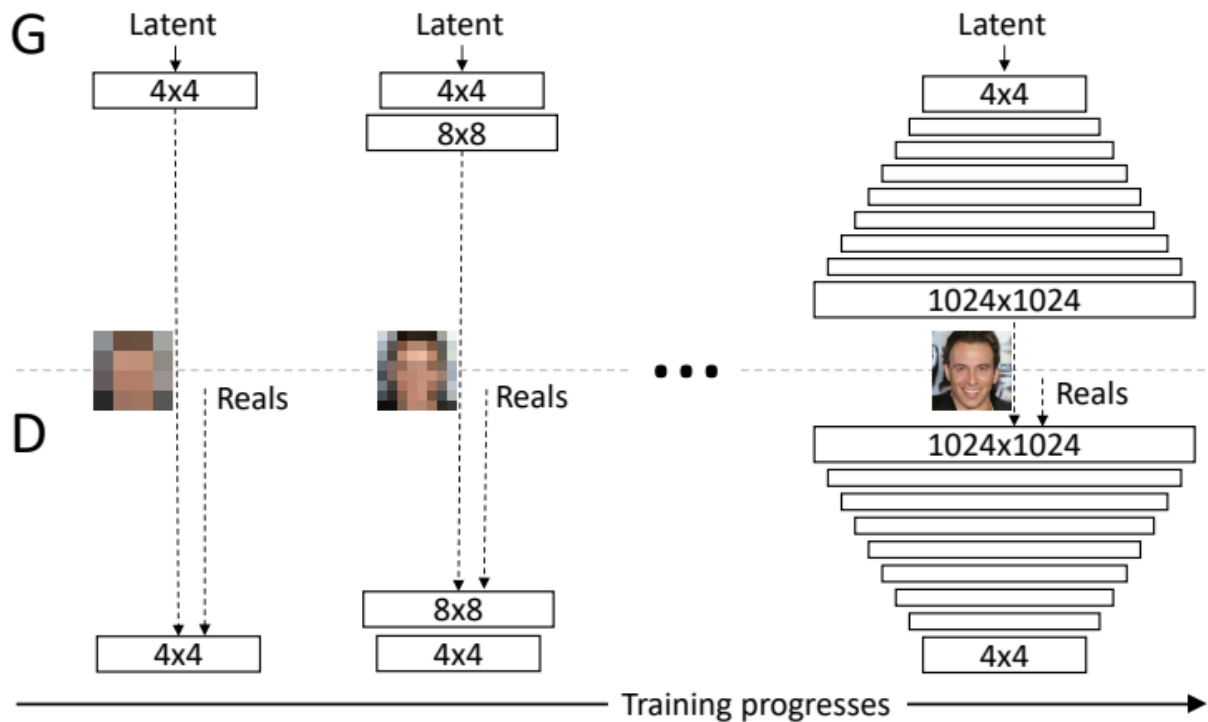


Figure 3.6: Progressive growing architecture of ProGAN.

The main architecture of ProGAN features a specialized generator and discriminator.

Generator:

1. PixelNorm: Normalizes the input noise vector to have unit length and upsamples the noise vector to 4×4 feature maps.
2. LeakyReLU: Activation function introducing non-linearity.
3. WConv2d: Weight-scaled convolutional layer with LeakyReLU activation, enhancing training stability.
4. Progressive Blocks (ConvBlock, WConv2d): Repeated convolutional blocks with weight-scaled convolutions and LeakyReLU activation for progressive upscaling of feature maps.
5. Fade-In Function (Linear interpolation): Gradual transition between upscaled and generated images using linear interpolation.

6. Forward Function (Initial block, Progressive blocks, Fade-In): Takes input noise vector, fade-in alpha, and number of progressive steps. Passes through initial block, iteratively through progressive blocks with upsampling, and applies fade-in between blocks.

Discriminator:

1. Progressive Blocks: Repeated convolutional blocks with weight-scaled convolutions and LeakyReLU activation for progressive downsampling of feature maps.
2. Initial RGB Layer (WSConv2d): Converts input images to feature maps using weight-scaled convolutions.
3. Average Pooling: Downsamples the feature maps using average pooling.
4. Final Block (Convolutional layers, WSConv2d): Concludes with convolutional layers and a weight-scaled convolution for final classification.
5. Minibatch Standard Deviation: Concatenates batch statistics to provide information about batch/image variation, aiding discrimination.
6. Fade-In Function (Linear interpolation): Gradual transition between downscaled and input images using linear interpolation.
7. Forward Function (Initial RGB Layer, Progressive blocks, Fade-In): Takes input image, fade-in alpha, and number of progressive steps. Converts to feature maps, iteratively passes through progressive blocks with downsampling, and applies fade-in between blocks. Returns final classification result.

The objective function of the Progressive Generative Adversarial Network (ProGAN) can be expressed through equation 2.

$$\min_G \max_D V_{ProGAN}(D, G) = E_{x \sim P_{data}(x)}[\log D(x)] + E_{z \sim P_z(z)}[\log(1 - D(G(z)))] \quad (2)$$

In this equation, we have different parts that represent the ProGAN model. The generator network G makes fake data $G(z)$ from random noise z , while the discriminator network D tries to tell if data is real x or fake $G(z)$. The $P_{data}(x)$ and $P_z(z)$ parts show where the real data and noise come from. The $V_{ProGAN}(D, G)$ part is the main thing we're trying to make smaller for the generator and bigger for the discriminator during training. This whole equation helps ProGAN learn to make fake data that looks real and helps the discriminator learn to tell the difference between real and fake data by giving it harder and harder examples.

ProGAN is an innovative approach to training Generative Adversarial Networks (GANs) that revolutionizes how image resolution is increased during training. It introduces incremental

growth, where both the generator and discriminator evolve together, focusing first on capturing large-scale structures before fine details. This method ensures smoother transitions as new layers are added, reducing disruptions in training. By primarily training at lower resolutions, ProGAN significantly reduces training time while still achieving faster convergence. Unlike hierarchical approaches, ProGAN utilizes a single GAN architecture, simplifying the training process while effectively capturing complex image mappings. The incorporation of normalization techniques, such as equalized learning rates and pixelwise feature vector normalization, stabilizes training dynamics. ProGAN also evaluates image quality and variation using multi-scale similarity measures, leading to improved convergence and reduced training time through progressive growing. Overall, ProGAN's methodology ensures stable training, faster convergence, and superior results for high-resolution GANs. Progressive growing architecture of ProGAN is displayed in Figure 3.6.

3.3 Experimental Setup

In our experimental setup, we integrate two well-established models to enhance traffic sign detection: You Only Look Once (YOLO) and Generative Adversarial Network (GAN) techniques. These models, developed by previous researchers, serve as foundational components in our approach to improving the accuracy and performance of traffic sign detection systems.

3.3.1 Image Dataset

This main dataset offers 7000 high-resolution images of 200 Slovenian traffic sign categories. Collected via a vehicle-mounted camera, it covers urban and rural areas, with images selected only if they contain traffic signs. The "DFG Traffic Sign Dataset" focuses on planar signs, comprising 5254 training images, 1703 testing images, and distinct resolution variations [39]. I've chosen 1176 images of 5 traffic sign categories for model enhancement, with an 80-20 split for training and testing. This forms 978 training images with 1028 traffic sign instances and 198 testing images with 234 instances of traffic sign. The selected traffic sign categories are shown in Figure 3.7. The number of instances for different traffic sign categories are represented in Table 3.1.

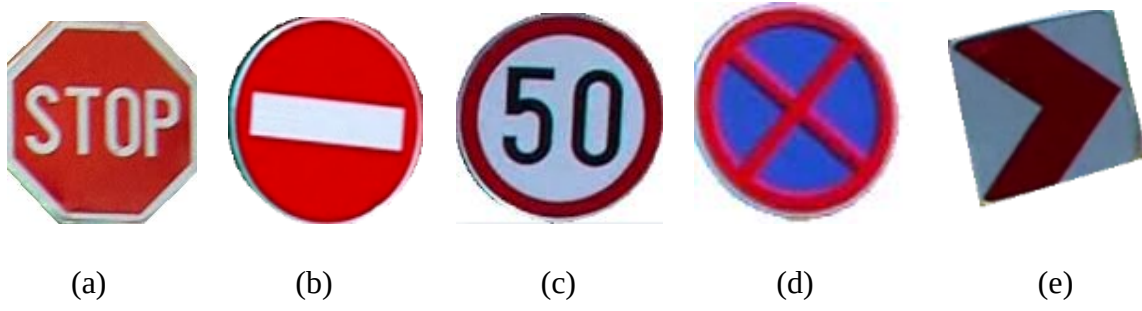


Figure 3.7: Selected dataset categories (a) II-2, (b) II-4, (c) II-30-50, (d) II-34, (e) III-107.1-2.

Table 3.1: Traffic Sign Instances in the training dataset.

Class ID	Class Name	Instances
C1	II-2	230
C2	II-4	200
C3	II-30-50	200
C4	II-34	198
C5	III-107.1-2	200

3.3.2 Data Generation by LSGAN and ProGAN:

In this study, both LSGAN and ProGAN architectures are employed to augment dataset images. LSGAN, a Generative Adversarial Network variant leveraging least squares loss, is utilized for its ability to enhance training stability and improve image quality, offering advantages such as improved stability, faster convergence, and higher-quality image generation compared to traditional GANs.

For the LSGAN model, a learning rate of 0.0002, betas of (0.5, 0.999), and weight decay of 0.00001 are employed during training to generate 64×64 sized images. The training of LSGAN model is conducted upto 2000 epoch. The generator utilizes noise vector of size 100 as input, while the discriminator input images remain consistent with the output or generated images. Training data consist of cropped and resized traffic sign objects from main images.

In contrast, ProGAN, an advanced variant of GANs, progressively increases the resolution of generated images during training, thereby enabling the generation of high-quality, high-resolution images while maintaining stability. ProGAN offers progressive resolution growth during training, allowing for the generation of superior quality images compared to LSGAN.

Due to device limitations, ProGAN image generation is constrained to 64×64 dimensions in this study. ProGAN training employs a learning rate of 0.001, betas of (0.0, 0.99), and batch sizes of [32, 32, 32, 16, 16] for progressive epochs of [30, 30, 30, 30, 30]. The discriminator input is 64×64 in size, while the generator utilizes noise vector of dimension 256 for image generation. For each GAN network, 50 high-quality images are chosen per class, resulting in a total selection of 250 generated images across 5 classes.

3.3.3 Multi-scale Image Generation

Multi-scale images in a dataset refer to images available at different resolutions or sizes, providing a broader range of features and details. Multi-scale images are necessary to make a model robust for handling images of different sizes effectively. It is easy to compress image to smaller size by simply resizing the images. In this work we compress our images generated from GAN network from 64×64 dimension to 32×32 sized. However, generating higher-resolution images from low-resolution ones poses a challenge. To address this challenge, two different-shaped EDSR (Enhanced Deep Super-Resolution) GAN networks are employed. One model doubles the size, while the other model quadruples the size of the images. Both models are trained separately on images generated by LSGAN and ProGAN to generate images of 128×128 and 256×256 sizes in each case. For training the EDSR network, parameters including a learning rate of 0.0001, weight decay of 0.00001, and gamma of 0.2 are utilized. The model consists of 16 residual blocks to generate high-resolution color images, and the training is conducted for 200 epochs. The combination of LSGAN and ProGAN generated for different scale and different class are represented in Table 3.2.

Table 3.2: Combination of GAN generated traffic sign images.

GAN	Dimension	C1	C2	C3	C4	C4	Total Images
LSGAN	32×32	50	50	50	50	50	1000
	64×64	50	50	50	50	50	
	128×128	50	50	50	50	50	
	256×256	50	50	50	50	50	
ProGAN	32×32	50	50	50	50	50	1000
	64×64	50	50	50	50	50	
	128×128	50	50	50	50	50	
	256×256	50	50	50	50	50	

3.3.4 Formation of training dataset

In this work four different sized images are generated for both LSGAN and ProGAN generated images and also for every traffic sign category. Total of 1000 images are generated for each GAN network. These 1000 multi-scaled images contain only traffic sign. For creating training images 100 real images of varied condition are selected and these 1000 images are paste on the 100 images randomly. Before pasting the generated 1000 images of traffic signs, the traffic signs located in 100 real images are covered. In this way we generate 100 training images differently for LSGAN and ProGAN generated images. Table 3.3 shows the training dataset combination for this work.

Table 3.3: Training dataset combination.

Group	Dataset	No of Training Images
1	Original Images	978
2	Original Images + LSGAN Generated Images	978+100
3	Original Images + ProGAN Generated Images	978+100

3.3.5 YOLOv4 Model Setup

The YOLO model configuration provided outlines key parameters crucial for training and inference. With a batch size and subdivisions set to 64, the model processes images efficiently during training. Input images are standardized to 416×416 pixels with three RGB channels.

Training parameters like momentum 0.949, decay 0.0005, and augmentation factors including saturation 1.5, exposure 1.5, and hue 0.1 are finely tuned to enhance model learning. A low learning rate of 0.001 is utilized along with a burn-in period of 1000 and a maximum of 10000 batches trained with a stepwise policy, employing steps at 8000 and 9000 with scales of 0.1. The mosaic technique is enabled for data augmentation. The model architecture includes the Mish activation function and varies filters across layers, employing down sampling through convolutional layers with a stride of 2 and incorporating route and shortcut connections for feature concatenation. The model integrates the Spatial Pyramid Pooling (SPP) module for capturing multi-scale information effectively. YOLO layers are configured with three layers, predefined anchor boxes, and five classes. Parameters such as jitter 0.3, ignore threshold 0.7, truth threshold 1, and IOU threshold 0.213 are set to optimize object detection accuracy. Additionally, class and IOU normalizers, IOU loss function, and NMS parameters contribute to the model's robustness in detecting objects within images. Adjusting these parameters plays a significant role in optimizing the model's performance for object detection tasks.

3.3.6 YOLOv5 Model Setup

In the presented YOLOv5 training setup, the model underwent training for 100 epochs employing the YOLOv5m architecture, utilizing a batch size of 16 and images resized to dimensions of 416×416 pixels. The Stochastic Gradient Descent (SGD) optimizer was employed, with initial and final learning rates lr_0 and lr_f both set at 0.01, accompanied by a momentum of 0.937 and weight decay of 0.0005. The warm-up period spanned 3 epochs, characterized by a warm-up momentum of 0.8 and a bias learning rate of 0.1. Augmentation strategies included blur, median blur, grayscale conversion, and Contrast Limited Adaptive Histogram Equalization (CLAHE), each applied with a conservative probability. Furthermore, geometric transformations such as translation, scaling, and flipping were integrated, with specified probabilities. These parameters collectively govern the training dynamics and ultimately influence the performance of the YOLOv5m model in object detection tasks across the designated dataset.

3.3.7 System Overview

Figure 3.8 shows the brief overview of the methodology used in this work. This including synthetic image generation, multi-scale image generation and training with YOLO and detection.

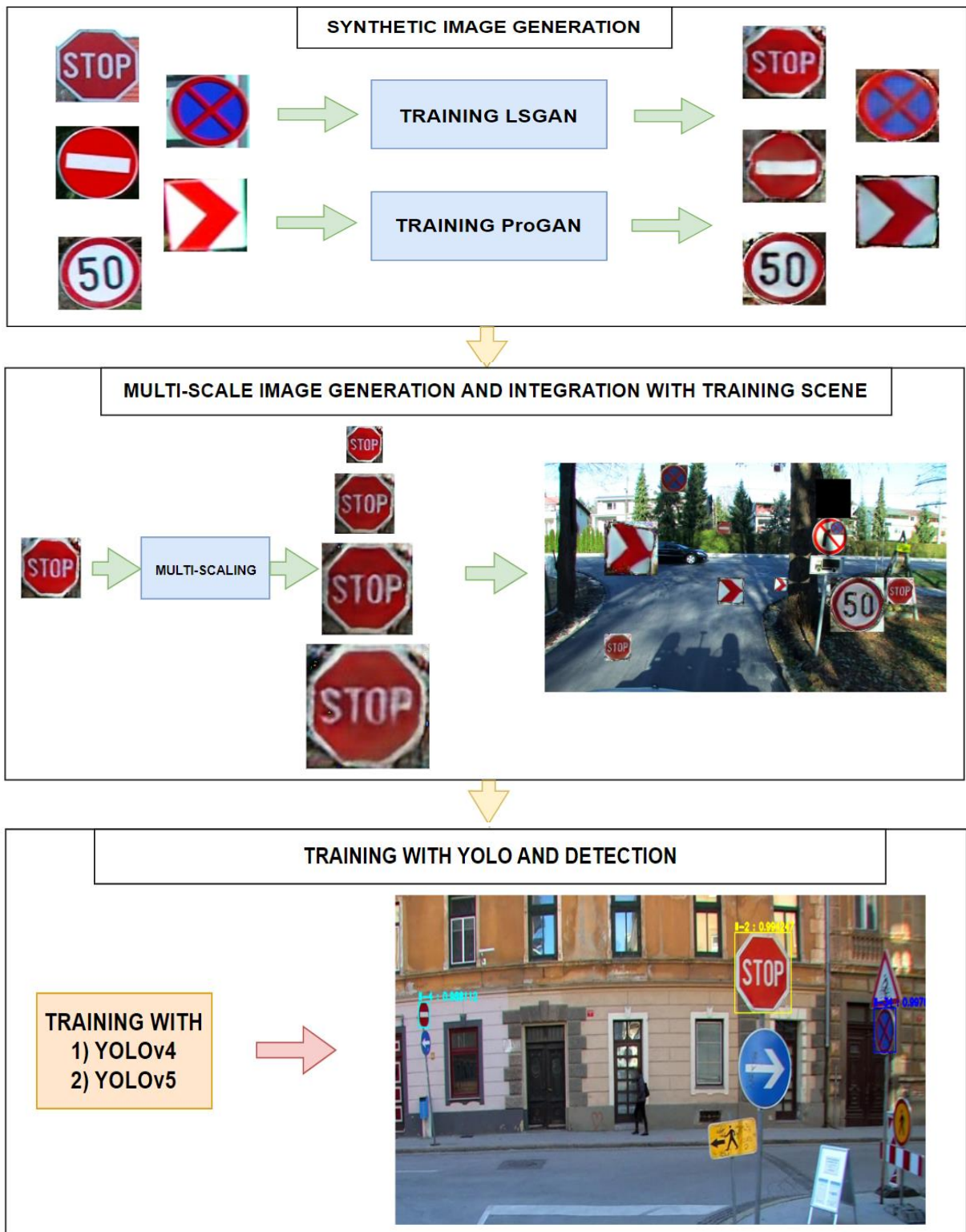


Figure 3.8: An overview of the system.

Chapter 4

Result Analysis

4.1 Data Generation Result

Our study assessed the data generation abilities of multiple GANs by examining how closely the generated images matched real photos. We utilized three distinct evaluation metrics: SSIM (Structural Similarity Index Measure), MSE (Mean Squared Error), and PSNR (Peak Signal-to-Noise Ratio).

Mean Squared Error (MSE) is a common metric used to measure the average squared difference between the pixel values of two images. It is calculated by taking the average of the squared differences between corresponding pixel values in the generated image and the reference (real) image. A lower MSE value indicates greater similarity between the images.

Equation 3 presents the mathematical formulation for the Mean Squared Error (MSE).

$$MSE = \frac{\sum_{i=1}^n (P_i - Q_i)^2}{n} \quad (3)$$

Where n is the total number of pixels in the image, P_i is the pixel value of the reference image at position i and Q_i is the pixel value of the generated image at position i .

MSE is used to compare GAN-generated images because it provides a quantitative measure of the overall difference between the generated and reference images. A lower MSE value indicates that the generated image is closer to the reference image in terms of pixel values.

Peak Signal-to-Noise Ratio (PSNR) is another widely used metric for evaluating image quality. It measures the ratio between the maximum possible power of a signal and the power of corrupting noise that affects the fidelity of its representation. In the context of image processing, PSNR is used to quantify the quality of a generated image compared to the original image.

Equation 4 depicts the calculation method for the Peak Signal to Noise Ratio (PSNR).

$$PSNR = 10 \times \log \log_{10} \frac{MAX^2}{MSE} \quad (4)$$

Where MAX is the maximum possible pixel value of the image (e.g., 255 for an 8-bit image), MSE is the Mean Squared Error between the reference and generated images.

PSNR is used to compare GAN-generated images because it provides a quantitative measure of the quality of the generated image compared to the reference image. A higher PSNR value indicates higher image fidelity and less perceptible differences between the generated and reference images. Therefore, PSNR is commonly used as a perceptual quality measure in image processing applications.

SSIM stands for Structural Similarity Index Measure. It is a metric used to quantify the similarity between two images, typically a reference image and a distorted image. SSIM is widely used in image processing and computer vision tasks to assess the quality of images and compare the perceptual similarity between them.

SSIM compares three aspects of the images: luminance, contrast, and structure. It computes similarity scores for each aspect and combines them to produce an overall similarity index. The SSIM index ranges from -1 to 1, where a value of 1 indicates perfect similarity between the images.

The equation shown as Equation 5 outlines the mathematical description of the SSIM.

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (5)$$

Where μ_x and μ_y are the average luminance values and σ_x^2 and σ_y^2 are the variances of the luminance values of the reference image x and the distorted image y, respectively. σ_{xy} is the covariance of the luminance values of the reference image x and the distorted image y. C_1 and C_2 are constants.

SSIM is used to compare GAN generated images because it provides a more comprehensive measure of image similarity compared to traditional metrics like mean squared error (MSE) or peak signal-to-noise ratio (PSNR). SSIM takes into account not only pixel-wise differences but also structural information and perceptual features, making it well-suited for assessing the quality of generated images in GANs. By using SSIM, researchers can better evaluate the visual fidelity and perceptual similarity of GAN generated images to the ground truth images, which is crucial in tasks such as image generation, image super-resolution, and image restoration.

We looked at the images created by two different methods: LSGAN and ProGAN, and compared them to the original images.

First, we measured how close the generated images were to the originals using something called Mean Squared Error (MSE). LSGAN images had an average MSE of 6492, while ProGAN

images had an average MSE of 6675. This means that, on average, LSGAN images were slightly closer to the originals in terms of pixel values.

Next, we looked at another measure called Peak Signal to Noise Ratio (PSNR). It tells us how much noise is in the images. Interestingly, LSGAN images had a higher PSNR than ProGAN images, which means they had less noise.

But that's not the whole story. We also looked at something called Structural Similarity (SSIM), which is about how similar the structures or shapes in the images are. ProGAN images scored higher in SSIM compared to LSGAN images. In simple terms, ProGAN images had shapes and structures more similar to the original images.

Breaking it down further:

- For ProGAN images, class C1 had the highest structural similarity with a score of 0.401, while class C5 had the lowest with 0.273.
- For LSGAN images, class C5 had the highest structural similarity with a score of 0.336, while class C4 had the lowest with 0.246.

So, in summary, while LSGAN images were closer to the original images in terms of pixel values and had less noise, ProGAN images had shapes and structures that were more similar to the originals. Table 4.1, 4.2 and 4.3 shows LSGAN and ProGAN generated images performance evaluation using MSE, PSNR and SSIM algorithm respectively.

Table 4.1: Different GAN performance evaluation using MSE.

Class Name	LSGAN MSE	ProGAN MSE
C1	5923	4940
C2	6814	7148
C3	8032	7371
C4	5343	5399
C5	6348	8518
Average	6492	6675

Table 4.2: Different GAN performance evaluation using PSNR.

Class Name	LSGAN PSNR (dB)	ProGAN PSNR (dB)
C1	10.405	11.194
C2	9.797	9.588
C3	9.082	9.455
C4	10.853	10.809
C5	10.105	8.828
Average	10.048	9.975

Table 4.3: Different GAN performance evaluation using SSIM.

Class Name	LSGAN SSIM	ProGAN SSIM
C1	0.322	0.401
C2	0.288	0.301
C3	0.328	0.321
C4	0.246	0.298
C5	0.336	0.273
Average	0.304	0.319



Figure 4.1: Original and generated images (a) Original Images, (b) LSGAN generated images, and (c) ProGAN generated images.

Figure 4.1 shows the comparison between GAN generated images with the original images. Figure 4.1 (a) represents the original images. Figure 4.1 (b) and (c) shows the LSGAN and ProGAN generated images respectively.

We utilized two types of Generative Adversarial Networks (GANs) to produce images sized at 64×64 pixels. To enhance the models' performance, we generated multi-scale images. Initially, we resized the GAN-generated images to 32×32 pixels using a manual compression technique. Subsequently, employing the EDSR GAN, we upscaled GAN generated 64×64 images to higher resolutions, resulting in two distinct sizes: 128×128 pixels and 256×256 pixels. This process yielded four sets of differently scaled images. These sets were generated separately for LSGAN and ProGAN, each comprising 1000 images featuring only traffic signs.

Figure 4.2 presents a comparative analysis of the four differently scaled LSGAN-generated images for each traffic sign class, while Figure 4.3 showcases the same comparison for ProGAN-generated images.

Finally, the multi-scaled images containing only traffic signs from various classes were incorporated randomly into 100 training image scenes. Before incorporating generated images into the main training images, we removed the original traffic signs from the images. This integration was executed differently for LSGAN and ProGAN, resulting in the creation of Figure 4.4 images.



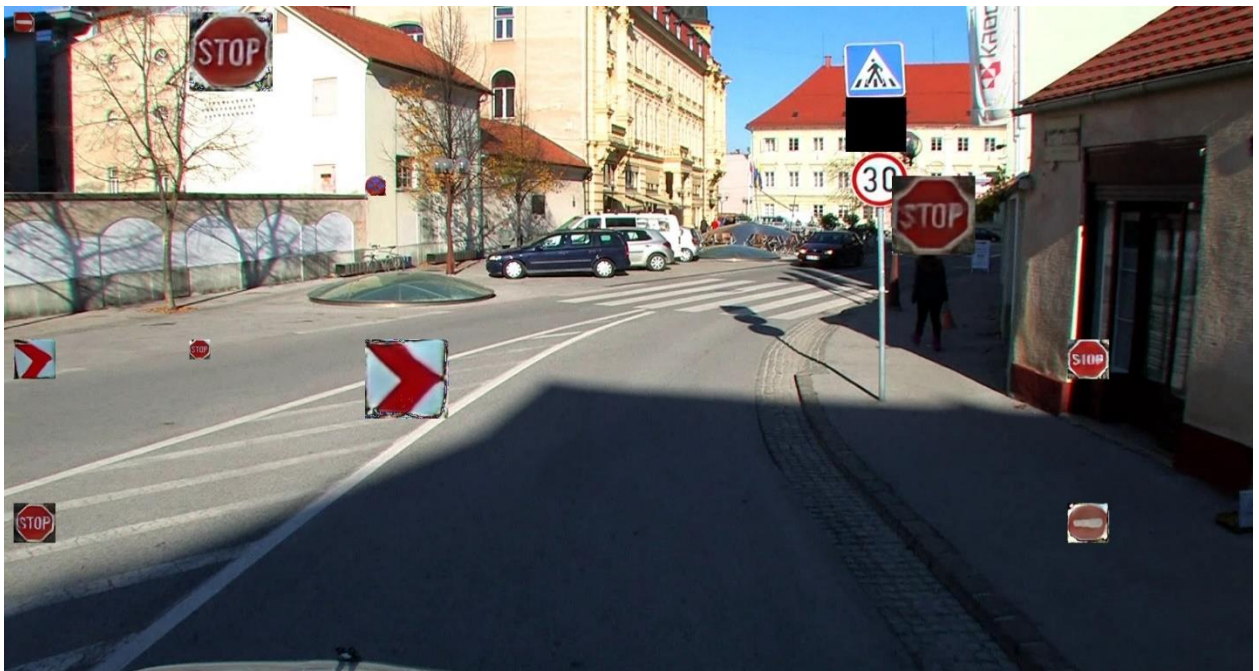
Figure 4.2: LSGAN generated images with size (a) 32×32,
(b) 64×64, (c) 128×128, and (d) 256×256.



Figure 4.3: ProGAN generated images with size (a) 32×32, (b) 64×64, (c) 128×128, and (d) 256×256.



(a)



(b)

Figure 4.4: (a) LSGAN and (b) ProGAN generated images integrated on real training image scenes.

4.2 YOLOv4 and YOLOv5 Training Results

A comprehensive evaluation of object detection models YOLOv4 and YOLOv5 on three distinct datasets reveals intriguing insights into their performance variances. By comparing precision, recall, F1-score, and mean Average Precision (mAP) across these datasets, a nuanced understanding emerges regarding the impact of dataset composition and model architecture on detection accuracy.

4.2.1 Performance Metrics

When evaluating the results obtained from YOLOv4 and YOLOv5 for object detection tasks, a range of comprehensive metrics is employed to provide a thorough assessment of their performance. These metrics encompass various aspects of detection accuracy and precision, offering detailed insights into the model's capabilities.

In traffic sign detection, TP (True Positives) are instances where the system correctly identifies a traffic sign, like a stop sign, in an image. FN (False Negatives) occur when a stop sign is present but the system fails to detect it. FP (False Positives) happen when the system incorrectly identifies something as a stop sign when it's not, like mistaking a round object for a stop sign. TN (True Negatives) are when the system correctly identifies that there is no stop sign when indeed there isn't one in the image. These metrics help assess the accuracy and reliability of a traffic sign detection system by showing how often it correctly identifies signs and how often it makes mistakes.

Precision, Recall, and the F1-score are fundamental metrics used to evaluate the performance of object detection models. Precision is a measure of the accuracy of a model's positive predictions. It quantifies the proportion of correctly identified positive instances out of all instances that the model predicts as positive. In Equation 6, Precision is defined using mathematical notation.

$$Precision = \frac{TP}{TP+FP} \quad (6)$$

Recall measures the model's ability to capture all relevant instances of a class within the dataset. It calculates the proportion of correctly identified positive instances (True Positives) out of all actual positive instances. Equation 7 illustrates the mathematical depiction of Recall.

$$Recall = \frac{TP}{TP+FN} \quad (7)$$

The F1-score serves as a balanced evaluation metric, synthesizing both Precision and Recall into a single value. It is computed as the harmonic mean of Precision and Recall, emphasizing a trade-off between the two metrics. The F1-score is represented mathematically in Equation 8.

$$F1 - score = \frac{2 \times (Precision \times Recall)}{Precision + Recall} \quad (8)$$

Intersection over Union (IoU) is a metric used in object detection tasks to evaluate the accuracy of bounding box predictions. It measures the overlap between the predicted bounding box and the ground truth bounding box of an object in an image. The IoU is calculated by dividing the area of intersection between the two bounding boxes by the area of their union. Mathematically, the IoU is expressed in equation 9.

$$IoU = \frac{Area\ of\ Intersection}{Area\ of\ Union} \quad (9)$$

IoU is essential in object detection as it provides a quantitative measure of how well the predicted bounding box aligns with the ground truth bounding box. Higher IoU values indicate better alignment and accuracy of the object detection model's predictions. It is commonly used as a criterion for evaluating the performance of object detection algorithms and is often incorporated into loss functions during training to optimize the model's ability to produce accurate bounding box predictions.

Average Precision (AP) is a metric used in object detection to assess the accuracy of the model's predictions across different classes or categories. It calculates the area under the precision-recall curve, which graphically represents the trade-off between precision (the fraction of correct positive predictions out of all positive predictions) and recall (the fraction of relevant instances retrieved out of all relevant instances). AP is computed by integrating the precision-recall curve and is a measure of how well the model balances precision and recall across various thresholds for classifying positive detections.

Mean Average Precision (mAP) is the average of the AP values calculated for multiple classes or categories in an object detection task. It provides an overall measure of the model's performance across all classes by taking into account the AP values for each class and averaging them. mAP is commonly used to evaluate the effectiveness of object detection models in detecting objects across the entire dataset, providing a single aggregated metric for comparison across different models and datasets.

The loss function in YOLO is crucial as it guides the training process by quantifying the model's performance. It ensures the model learns to accurately predict object locations and confidence scores, ultimately improving object detection accuracy and it represented in equation 10.

$$\begin{aligned} & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{obj} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + \right. \\ & \left. (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{obj} (C_i - \hat{C}_i)^2 + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{noobj} (C_i - \hat{C}_i)^2 + \\ & \sum_{i=0}^{S^2} \mathbb{I}_i^{obj} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \end{aligned} \quad (10)$$

In the YOLO algorithm, the loss function is defined by several key components. The coefficient λ_{coord} is responsible for balancing the localization loss, which measures the accuracy of predicted bounding box coordinates, while λ_{noobj} regulates the confidence score loss, evaluating the model's ability to predict object presence with certainty. The equation operates over a grid of S^2 cells, with each cell potentially containing B predicted bounding boxes. Indicator functions

$\mathbb{I}_{ij}^{obj}$ and $\mathbb{I}_{ij}^{noobj}$ denote the presence or absence of objects within cells and their associated bounding boxes. Center coordinates (x_i, y_i) and dimensions (w_i, h_i) represent the predicted bounding boxes, while $(\hat{x}_i, \hat{y}_i)$ and $(\hat{w}_i, \hat{h}_i)$ denote the ground truth bounding boxes. Confidence scores C_i and $\hat{C}_i$ assess the model's certainty in its predictions, and class probabilities $p_i(c)$ and

$\hat{p}_i(c)$ evaluate the accuracy of predicted class labels. Together, these components form the foundation of the loss function, guiding the optimization process to improve object detection performance in YOLO.

4.2.2 Performance Comparison

Comparing the performance metrics across the different models and datasets reveals notable differences in their object detection capabilities. The YOLOv5 model trained on Group 2 dataset exhibits the highest precision of 0.934, recall of 0.747, F1-score of 0.84, and mean Average Precision (mAP) of 81%. These metrics indicate its superior ability to accurately detect and localize objects compared to other configurations.

The YOLOv4 model trained on Group 3 dataset also demonstrates strong performance, with a precision of 0.92, recall of 0.744, F1-score of 0.84, and mAP of 80.80%. While slightly lower than the YOLOv5 model on Group 2, these metrics still position it as a robust performer, particularly in terms of precision and mAP.

In contrast, the YOLOv4 model trained on Group 1 dataset shows comparatively lower performance metrics, with a precision of 0.87, recall of 0.71, F1-score of 0.78, and mAP of 79.99%. These values suggest that using only the original training images without augmentation may limit the model's ability to accurately detect objects.

Similarly, the YOLOv5 model trained on Group 1 dataset also exhibits lower performance metrics compared to its counterparts trained on augmented datasets, with a precision of 0.908, recall of 0.73, F1-score of 0.80, and mAP of 78.80%.

Overall, the inclusion of generated images in the training datasets (Groups 2 and 3) leads to improved performance in object detection tasks, with the YOLOv5 model trained on Group 2 dataset achieving the highest overall performance based on precision, recall, F1-score, and mAP. The YOLOv4 model trained on Group 3 dataset also demonstrates competitive performance, particularly in terms of precision and mAP, highlighting the efficacy of incorporating ProGAN generated images in the training process.

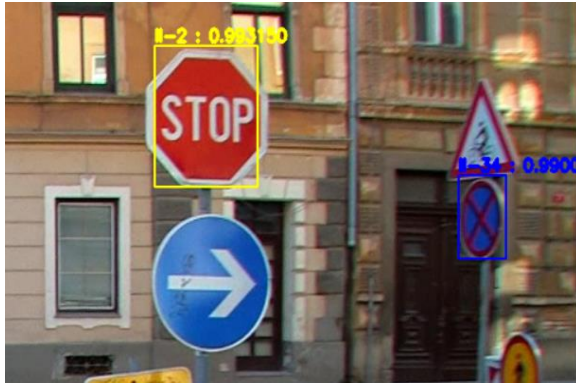
Table 4.4: Validation results of YOLOv4 on different datasets.

Dataset	Class Name	AP (%)	TP	FP	FN	Recall	Precision	IoU (%)	F1-score	mAP (%)
Group1	C1	72.02	166	25	69	0.71	0.87	71.83	0.78	79.99
	C2	71.42								
	C3	90.99								
	C4	85.49								
	C5	80.03								
Group2	C1	75.44	167	25	68	0.71	0.87	74.80	0.78	80.04
	C2	66.32								
	C3	91.22								
	C4	87.91								
	C5	79.34								
Group3	C1	74.77	171	22	64	0.73	0.89	77.05	0.80	80.21
	C2	67.14								
	C3	91.77								
	C4	88.92								
	C5	78.45								

Table 4.5: Validation results of YOLOv5 on different datasets.

Dataset	Class Name	AP (%)	Recall	Precision	F1-score	mAP50 (%)	mAP50-95 (%)
Group1	C1	74.60	0.730	0.908	0.80	78.80	66.10
	C2	60.40					
	C3	86.90					
	C4	88.50					
	C5	83.80					
Group2	C1	72.70	0.747	0.934	0.84	81.00	67.90
	C2	68.60					
	C3	87.70					
	C4	92.00					
	C5	84.20					
Group3	C1	71.70	0.744	0.920	0.80	80.80	68.10
	C2	65.50					
	C3	92.40					
	C4	88.80					
	C5	85.80					

Table 4.4 and 4.5 shows validation results of YOLOv4 and YOLOv5 model on different datasets.



(a)



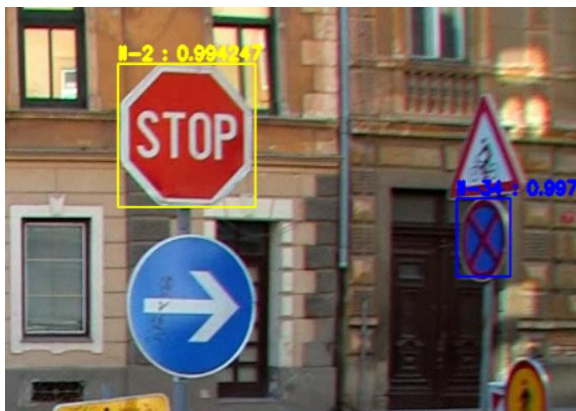
(d)



(b)



(e)



(c)



(f)

Figure 4.5: The detection result for (a) YOLOv4 and Group 1, (b) YOLOv4 and Group 2, and (c) YOLOv4 and Group 3 dataset and for (d) YOLOv5 and Group 1, (e) YOLOv5 and Group 2, and (f) YOLOv5 and Group 3 dataset.

4.2.3 User Interface

We've made a desktop software that spots traffic signs in real-time, perfect for devices with limited power using CPU-based inference. Right now, it's all about processing images, but we're working on adding video processing and device camera access for high-powered systems. We've used smart programming languages and libraries to make sure it works on lots of devices. It's super easy to use – just upload an image and get instant traffic sign detection results. While it's currently powered by the CPU, we're aiming to make it even faster by adding support for GPUs. This software is a big step forward in keeping roads safe, especially for devices that aren't super powerful, and we're excited to make it even better with GPU support and displayed in Figure 4.6.



Figure 4.6: User interface for detecting traffic sign in an image.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

In conclusion, our study delved into enhancing traffic sign detection systems through the integration of Generative Adversarial Networks (GANs) and multi-scale training techniques. We evaluated the effectiveness of utilizing LSGAN and ProGAN to expand datasets and improve object detection robustness for various image sizes.

Our findings indicate that while LSGAN-generated images were closer to the original images in terms of pixel values and had less noise, ProGAN-generated images exhibited shapes and structures more similar to the originals. This suggests that ProGAN may be more suitable for generating images conducive to traffic sign detection tasks.

Furthermore, our comparison of different models and datasets highlights the significant impact of including generated images in training datasets for object detection tasks. The YOLOv5 model trained on original and LSGAN generated dataset showed the highest overall performance, emphasizing the importance of augmentation for improved precision, recall, F1-score, and mean Average Precision (mAP). Additionally, the YOLOv4 model trained on original and ProGAN dataset also demonstrated competitive performance, underscoring the efficacy of incorporating ProGAN-generated images in the training process to enhance object detection capabilities. Overall, our findings emphasize the value of utilizing augmented datasets for enhancing the accuracy of object detection models in traffic sign detection systems.

Overall, our research underscores the significance of leveraging GANs for dataset expansion and employing multi-scale training techniques to enhance the efficiency and accuracy of traffic sign detection algorithms, ultimately contributing to improved road safety and traffic management systems.

5.2 Future Work

For further enhancing traffic sign detection and utilizing YOLO and GAN-based performance boosts, several potential future avenues can be explored.

- Investigate other GAN architectures beyond LSGAN and ProGAN to enhance the generation of high-quality training images.
- Explore advanced super-resolution techniques beyond EDSR GAN to further improve image quality and model robustness.
- Experiment with diverse data augmentation strategies, including geometric transformations and occlusion patterns, to enhance dataset diversity and model generalization.
- Create adaptive algorithms to flexibly respond to varying environmental factors, bolstering traffic sign detection's adaptability and robustness in real-world conditions.
- Research integration of traffic sign detection systems with autonomous vehicles to contribute to road safety and transportation system efficiency.

References

- [1] L. Abdi and A. Meddeb, “Deep learning traffic sign detection, recognition and augmentation,” in *Proc. Symp. Appl. Comput.*, Apr. 2017, pp. 131–136.
- [2] R. Hasegawa, Y. Iwamoto, and Y.-W. Chen, “Robust detection and recognition of japanese traffic sign in the complex scenes based on deep learning,” in *Proc. IEEE 8th Global Conf. Consum. Electron. (GCCE)*, Oct. 2019, pp. 575–578.
- [3] Á. Arcos-García, J. A. Álvarez-García, and L. M. Soria-Morillo, “Evaluation of deep neural networks for traffic sign detection systems,” *Neurocomputing*, vol. 316, pp. 332–344, Nov. 2018.
- [4] H. Luo, Y. Yang, B. Tong, F. Wu, and B. Fan, “Traffic sign recognition using a multi-task convolutional neural network,” *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 4, pp. 1100–1111, Apr. 2018.
- [5] S. Song, Z. Que, J. Hou, S. Du, and Y. Song, “An efficient convolutional neural network for small traffic sign detection,” *J. Syst. Archit.*, vol. 97, pp. 269–277, Aug. 2019.
- [6] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards realtime object detection with region proposal networks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.
- [7] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single shot multibox detector,” in *Proc. Eur. Conf. Comput. Vis.*, in *Lecture Notes in Computer Science: Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics*, 2016, pp. 21–37.
- [8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.
- [9] Q. Xu, R. Lin, H. Yue, H. Huang, Y. Yang, and Z. Yao, “Research on small target detection in driving scenarios based on improved YOLO network,” *IEEE Access*, vol. 8, pp. 27574–27583, 2020.

- [10] J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” in Proc. 30th IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Jul. 2017, pp. 6517–6525.
- [11] J. Tao, H. Wang, X. Zhang, X. Li, and H. Yang, “An object detection system based on YOLO in traffic scene,” in Proc. 6th Int. Conf. Comput. Sci. Netw. Technol. (ICCSNT), Oct. 2017, pp. 315–319.
- [12] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. *ArXiv, abs/1804.02767*.
- [13] L. Zhao and S. Li, “Object detection algorithm based on improved YOLOv3,” *Electronics*, vol. 9, no. 3, p. 537, Mar. 2020.
- [14] Bochkovskiy, A., Wang, C., & Liao, H.M. (2020). “YOLOv4: Optimal Speed and Accuracy of Object Detection.” *ArXiv, abs/2004.10934*.
- [15] D. Bau, J.-Y. Zhu, H. Strobelt, B. Zhou, J. B. Tenenbaum, W. T. Freeman, and A. Torralba, “GaN dissection: Visualizing and understanding generative adversarial networks,” in Proc. 7th Int. Conf. Learn. Represent. (ICLR), 2019, pp. 1–18.
- [16] Y.-J. Cao, L.-L. Jia, Y.-X. Chen, N. Lin, C. Yang, B. Zhang, Z. Liu, X.-X. Li, and H.-H. Dai, “Recent advances of generative adversarial networks in computer vision,” *IEEE Access*, vol. 7, pp. 14985–15006, 2019.
- [17] C. Dewi, R.-C. Chen, Y.-T. Liu, and S.-K. Tai, “Synthetic data generation using DCGAN for improved traffic sign recognition,” *Neural Comput. Appl.*, vol. 33, no. 8, pp. 1–15, 2021.
- [18] W. Wang, C. Wang, T. Cui, and Y. Li, “Study of restrained network structures for wasserstein generative adversarial networks (WGANs) on numeric data augmentation,” *IEEE Access*, vol. 8, pp. 89812–89821, 2020.
- [19] G. Wei, M. Luo, H. Liu, D. Zhang, and Q. Zheng, “Progressive generative adversarial networks with reliable sample identification,” *Pattern Recognit. Lett.*, vol. 130, pp. 91–98, Feb. 2020.
- [20] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.

- [21] Mao, Xudong et al. "Least Squares Generative Adversarial Networks." *2017 IEEE International Conference on Computer Vision (ICCV)* (2016): 2813-2821.
- [22] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley, "On the effectiveness of least squares generative adversarial networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 12, pp. 2947–2960, Dec. 2019.
- [23] Karras, T., Aila, T., Laine, S., & Lehtinen, J. (2017). "Progressive Growing of GANs for Improved Quality, Stability, and Variation." *ArXiv, abs/1710.10196*.
- [24] Lim, Bee et al. "Enhanced Deep Residual Networks for Single Image Super-Resolution." *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* (2017): 1132-1140.
- [25] H. Zhang et al., "Real-Time Detection Method for Small Traffic Signs Based on Yolov3," in *IEEE Access*, vol. 8, pp. 64145-64156, 2020, doi: 10.1109/ACCESS.2020.2984554.
- [26] X. Ma, K. Ji, B. Xiong, L. Zhang, S. Feng and G. Kuang, "Light-YOLOv4: An Edge-Device Oriented Target Detection Method for Remote Sensing Images," in *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 14, pp. 10808-10820, 2021, doi: 10.1109/JSTARS.2021.3120009.
- [27] C. Dewi, R. -C. Chen, Y. -T. Liu, X. Jiang and K. D. Hartomo, "Yolo V4 for Advanced Traffic Sign Recognition With Synthetic Training Data Generated by Various GAN," in *IEEE Access*, vol. 9, pp. 97228-97242, 2021, doi: 10.1109/ACCESS.2021.3094201.
- [28] L. Wang, K. Zhou, A. Chu, G. Wang and L. Wang, "An Improved Light-Weight Traffic Sign Recognition Algorithm Based on YOLOv4-Tiny," in *IEEE Access*, vol. 9, pp. 124963-124971, 2021, doi: 10.1109/ACCESS.2021.3109798.
- [29] Q. Wang, X. Li and M. Lu, "An Improved Traffic Sign Detection and Recognition Deep Model Based on YOLOv5," in *IEEE Access*, vol. 11, pp. 54679-54691, 2023, doi: 10.1109/ACCESS.2023.3281551.
- [30] T. Shi, W. Zhu and Y. Su, "Improved Light-Weight Target Detection Method Based on YOLOv5," in *IEEE Access*, vol. 11, pp. 38604-38613, 2023, doi: 10.1109/ACCESS.2023.3267965.

- [31] Y. -J. Cao et al., "Recent Advances of Generative Adversarial Networks in Computer Vision," in *IEEE Access*, vol. 7, pp. 14985-15006, 2019, doi: 10.1109/ACCESS.2018.2886814.
- [32] Q. Wu, Y. Chen and J. Meng, "DCGAN-Based Data Augmentation for Tomato Leaf Disease Identification," in *IEEE Access*, vol. 8, pp. 98716-98728, 2020, doi: 10.1109/ACCESS.2020.2997001.
- [33] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang and S. P. Smolley, "On the Effectiveness of Least Squares Generative Adversarial Networks," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 12, pp. 2947-2960, 1 Dec. 2019, doi: 10.1109/TPAMI.2018.2872043.
- [34] C. -T. Lin, S. -W. Huang, Y. -Y. Wu and S. -H. Lai, "GAN-Based Day-to-Night Image Style Transfer for Nighttime Vehicle Detection," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 2, pp. 951-963, Feb. 2021, doi: 10.1109/TITS.2019.2961679.
- [35] H. Luo, Q. Kong and F. Wu, "Traffic Sign Image Synthesis with Generative Adversarial Networks," *2018 24th International Conference on Pattern Recognition (ICPR)*, Beijing, China, 2018, pp. 2540-2545, doi: 10.1109/ICPR.2018.8545787.
- [36] J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, "The German Traffic Sign Recognition Benchmark: A multi-class classification competition," *The 2011 International Joint Conference on Neural Networks*, San Jose, CA, USA, 2011, pp. 1453-1460, doi: 10.1109/IJCNN.2011.6033395.
- [37] C. Dewi, R. C. Chen, and Y.-T. Liu, "Taiwan stop sign recognition with customize anchor," in *Proc. ICCMS*, Brisbane, QLD, Australia, Feb. 2020, pp. 51–55.
- [38] Z. Zhu, D. Liang, S. Zhang, X. Huang, B. Li, and S. Hu, "Traffic-sign detection and classification in the wild," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2110–2118.
- [39] D. Tabernik and D. Skočaj, "Deep Learning for Large-Scale Traffic-Sign Detection and Recognition," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 4, pp. 1427-1440, April 2020, doi: 10.1109/TITS.2019.2913588.